qsBye

折腾笔记[21]-使用rust推理onnx模型

摘要

使用纯血rust推理onnx模型识别mnist手写数字.

关键词

rust;onnx;mnist;nalgebra;

关键信息

项目地址:
[https://github.com/ByteIO/bye.orbslam3.rs/blob/dev1/crates/seekslam_examples/examples/mnist_onnx.rs]

```
# 图像处理
image = "0.25.6"
imageproc = "0.25.0"
# onnx运行时
wonnx = { version = "0.5.1", path = "./static/wonnx/wonnx" }
# 线性代数
nalgebra = { version = "0.33.2", features = ["rand"] }
# 多线程框架
tokio = { version = "1.44.1", features = ["full"] }
# 错误处理
anyhow = "1.0.97"
```

原理简介

wonnx简介

[https://github.com/webonnx/wonnx/tree/master]
一个完全用 Rust 编写的 WebGPU 加速的 ONNX 推理运行时，适用于原生和 Web 端。
A WebGPU-accelerated ONNX inference run-time written 100% in Rust, ready for native and the web.

实现

```
#![allow(unused)]

//! 使用wonnx (纯Rust) 加载MNIST的ONNX模型。

// 标准库
use std::collections::HashMap;
use std::convert::TryInto;
use std::path::Path;
use std::time::Instant;

// 图像处理
use image::{imageops::FilterType, ImageBuffer, Pixel, Rgb};

// ONNX运行时
use wonnx::utils::OutputTensor;

// 线性代数
use nalgebra::{DMatrix, DVector};

// 参数管理
async fn run() {
    // 执行模型推理并获取结果
    let probabilities = execute_gpu().await.unwrap();
    // 获取概型输出的第一个张量
    let (_, probabilities) = probabilities.into_iter().next().unwrap();
    // 将概型输出的张量转换为Vec<f32>
    let probabilities: Vec<f32> = probabilities.try_into().unwrap();
    // 打印概型输出的概率值
    println!("steps: {:#?}", probabilities);
    // 打印概率值的长度
    println!("steps: {:#?}", probabilities.len());

    // 将概率值与索引绑定并转换为向量
    let mut probabilities = probabilities.iter().enumerate().collect::<Vec<_>>();

    // 根据概率值对向量进行降序排序
    probabilities.sort_unstable_by(|a, b| b.1.partial_cmp(a.1).unwrap());

    // 打印推理结果（概率最高的索引）
    println!("Infered result: {}", probabilities[0].0);
}

// 硬件管理
async fn execute_gpu() -> Option<HashMap<String, OutputTensor>> {
    // 创建输入数据的HashMap
    let mut input_data = HashMap::new();

    // 加载图像数据
    let image = load_image();
    // 将图像数据插入到输入数据的HashMap中
    input_data.insert("Input3".to_string(), image.as_slice().into());

    // 构造模型路径
    // let model_path = Path::new(env!("CARGO_MANIFEST_DIR"))
    //     .join("../../assets/wonnx_data/models")
    //     .join("opt-mnist.onnx");
    let model_path = Path::new("../../assets/wonnx_data/models").join("opt-mnist.onnx");
    // 加载ONNX模型
    let session = wonnx::Session::from_path(model_path).await.unwrap();

    // 记录推理前的时间
    let time_pre_compute = Instant::now();
    // 执行模型推理
    let result = session.run(&input_data).await.unwrap();
    // 记录推理后的时间
    let time_post_compute = Instant::now();
    // 打印推理时间
    println!(
        "time: post_compute: {:#?}",
        time_post_compute - time_pre_compute
    );
    // 返回模型推理结果
    Some(result)
}

// 使用tokio的main宏
#[tokio::main]
async fn main() {
    // 初始化日志
    // env_logger::init();
    // 记录主函数开始的时间
    let time_pre_compute = Instant::now();
    // 执行异步任务
    run().await;
    // 打印主函数的执行时间
    println!("time: main: {:#?}", time_pre_compute.elapsed());
}

// 加载图像数据
pub fn load_image() -> DMatrix<f32> {
    // 打开图像文件并将其转换为RGB格式，同时调整大小为28x28
    let image_buffer: ImageBuffer<Rgb<u8>, Vec<u8>> = image::open(
        Path::new("../../assets/wonnx_data/images")
            .join("7.jpg"),
    )
    .unwrap()
    .resize_exact(28, 28, FilterType::Nearest)
    .to_rgb8();

    // 构造一个形状为(1, 1, 28, 28)的矩阵
    let mut matrix = DMatrix::zeros(1, 784); // 28*28 = 784

    for j in 0..28 {
        for i in 0..28 {
            let pixel = image_buffer.get_pixel(i as u32, j as u32);
            let channels = pixel.channels();
            // 将像素值从[0, 255]范围归一化到[0, 1]范围
            let value = (channels[0] as f32) / 255.0; // 只取灰度值
            matrix[(0, j * 28 + i)] = value;
        }
    }

    matrix
}
```

效果

```
time: post_compute: 2.479209ms
steps: [
    -4.5589633,
    -2.6439452,
    6.0749683,
    6.473433,
    -4.836302,
    -0.36092412,
    -25.101564,
    18.573236,
    -2.43262,
    4.7025075,
]
steps: 10
Infered result: 7
time: main: 82.872875ms
```

分类: 折腾笔记

标签: Rust

好文要顶 关注我 收藏该文 微信分享

qsBye
粉丝 - 27 关注 - 0
会员号：107

0 推荐 0 反对

« 上一篇： 折腾笔记[20]-使用rust交换protobuf数据

posted @ 2025-04-08 04:58 qsBye 阅读(0) 评论(0) MD 编辑 收藏 举报

刷新评论 刷新页面 返回顶部

发表评论

编辑 预览

支持 Markdown

自动补全

提交评论 退出

[Ctrl+Enter快捷键提交]

【推荐】100%开源！大型工业跨平台软件C++源码提供，建模，组态！
【推荐】还在用 ECharts 开发大屏？试试这款永久免费的开源 BI 工具！
【推荐】国内首个AI IDE，深度理解中文开发场景，立即下载体验Trae
【推荐】编程新体验，更懂你的AI，立即体验豆包MarsCode编程助手
【推荐】轻量又高性能的 SSH 工具 IShell：AI 加持，快人一步

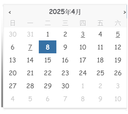
编辑推荐：
· 为什么构造函数需要尽可能的简单
· 探秘 MySQL 索引底层原理，解锁数据库优化的关键密码(下)
· 大模型 Token 究竟是啥：图解大模型Token
· 32岁程序员的中年求职记：四次键盘后的深度反思
· 继承的思维：从思维模式到架构设计的深度解析

阅读排行：
· 【保姆级教程】windows 安装 docker 全流程
· 基于Docker+DeepSeek+Dify ：搭建企业版本地私有化知识库超详细教程
· 由 MCP 官方推出的 C# SDK，使 .NET 应用程序、服务和策略够快速实现与 MCP 客户端
· 电商平台中订单未支付过期如何实现自动关单?
· X86-64位简易系统开发 - 从BIOS到程序始

Copyright © 2025 qsBye
Powered by .NET 9.0 on Kubernetes

公告

昵称： qsBye
园龄： 8年5个月
粉丝： 27
关注： 0



2025年4月

日	一	二	三	四	五	六
30	31	1	2	3	4	5
6	7	8	9	10	11	12
13	14	15	16	17	18	19
20	21	22	23	24	25	26
27	28	29	30	1	2	3
4	5	6	7	8	9	10

搜索

常用链接

我的随笔
我的评论
我的参与
最新评论
我的标签

我的标签

Rust(17)
SLAM(5)
线性代数(2)
LLM(1)

随笔分类

51单片机(10)
Android开发(18)
bouffalo单片机(1)
esp32单片机(24)
GNU Radio(4)
LLM精彩问答(7)
MATLAB(8)
Raspberry Pi单片机(6)
SCILAB(3)
stm32单片机(16)
TI单片机(3)
WebAssembly(3)
翻译(5)
高云FPGA(3)
公众号(0)
鸿蒙开发(3)
机器人(13)
前端开发(12)
嵌入式Linux(3)
人工智能(4)
运维相关(16)
折腾笔记(23)

随笔档案

2025年4月(4)
2025年3月(4)
2025年2月(6)
2025年1月(11)
2024年12月(1)
2024年10月(1)
2024年9月(4)
2024年8月(2)
2024年6月(3)
2024年5月(9)
2024年4月(3)
2024年3月(10)
2024年2月(8)
2024年1月(9)
2023年12月(4)
2023年11月(6)
2023年10月(4)
2023年9月(4)
2023年8月(10)
2023年7月(16)
2023年6月(2)
2023年5月(2)
2023年4月(3)
2023年3月(2)
2023年2月(7)
2023年1月(17)
2022年12月(11)
2022年11月(14)
2022年10月(8)
2022年9月(17)
更多

阅读排行榜

1. OpenWrt安装Filebrowser网页文件管理器(11691)
2. esp32笔记[5]-基于I2S协议实现音频播放(9585)
3. 该该NAS的硬盘SATA背板(6767)
4. 三星平板的笔记模板(比例:16:10)(4336)
5. 电犀牛R66x联盟曲体验(4291)

评论排行榜

1. 51单片机笔记[1]-基础实验(1)
2. zemax光学设计笔记[5]-ZPL或ZOS-API的简单使用(1)
3. 小米6安装Ubuntu Touch(1)

推荐排行榜

1. 折腾笔记[15]-配置Zed编辑器(1)
2. esp32笔记[5]-基于I2S协议实现音频播放(1)
3. esp32笔记[2]-blink闪灯(1)
4. stm32笔记[2]-HAL库安装2.4寸屏幕(1)
5. 基于esp32的MQTT开关门检测模块(1)

最新评论

1. Re:zemax光学设计笔记[5]-ZPL或ZOS-API的简单使用
请问博主用vsstudio运行的时候无法读取类型库文件(ZOSAPI.tlb)如何解决啊，ZOSAPI.tlb能用import读取，ZOSAPI.tlb读取却报错，研究了一天了，这两个文件在zosa...
--赤Xin

2. Re:小米6安装Ubuntu Touch
您好！请问应该如何扩容docker呢？
我按照本文给出的方案安装之后docker容量只有4G
--VantHoff

3. Re:51单片机笔记[1]-基础实验
大学买的单片机就没打开过.
--mossxzzom